Python Code Descriptions

Variable Switching DC Power Supply — Testing & Validation Framework

Table

#	Code Block	What It Does
1	Finding Your Instruments	Lists all VISA-connected devices
2	Generic Instrument Class	Base `BenchInstrument` with SCPI read/write
3	DC Power Supply	Controls Rigol DP832 for input voltage injection
4	Electronic Load	Programs constant-current load steps
5	Digital Multimeter	High-accuracy voltage/resistance readback
6	Oscilloscope	Captures waveforms, triggers, downloads raw data
7	STM32 Telemetry Reader	Parses real-time CSV from UART in a background thread
8	Firmware Telemetry (C)	Generates the CSV stream on the STM32 side
9	Load Regulation Test	Sweeps 0→5 A, logs data, plots regulation & efficiency
10	Transient Response	Triggers scope on load step, captures overshoot
11	OVP Protection Test	Injects fault via serial, verifies <20 µs shutdown
12	pytest Regression Suite	Formal pass/fail assertions for CI/CD
13	Report Generation	Auto-builds Word doc with tables and plots

1. Finding Your Instruments

Purpose: Discover all VISA-compatible instruments connected to the PC via USB, GPIB, or LAN.

This is the first script you run when setting up the test bench. It queries the PyVISA Resource Manager and prints a tuple of resource strings (e.g., USB0::vendor::model::serial::INSTR). Each string is a unique address you copy into your instrument wrapper constructors. Without this step, you cannot talk to any hardware.

2. Generic Instrument Class (BenchInstrument)

Purpose: Provide a reusable base class that abstracts SCPI command exchange for any bench instrument.

This class encapsulates four universal operations:

- __init__ — Opens the VISA resource and sets a 5-second command timeout.
- write() — Sends a SCPI command string (e.g., "OUTPUT1 ON") without expecting a reply.
- query() — Sends a command and returns the instrument response as a stripped string.
- idn() / reset() — Convenience wrappers around the IEEE-488 *IDN? identity query and *RST reset command.

All subsequent instrument-specific classes inherit from BenchInstrument, eliminating duplicated VISA boilerplate.

3. Programmable DC Power Supply (DCPowerSupply)

Purpose: Control a bench DC supply (e.g., Rigol DP832) to inject known voltages and currents into the power supply under test.

Methods exposed:

- set_voltage(channel, voltage) — Programs the output voltage on a specific channel.
- set_current(channel, current) — Sets the current limit for over-current protection during bring-up.
- output_on / output_off — Enables or disables the channel output.

- measure_voltage / measure_current — Reads back the actual output values using the supply's internal sense, useful for calculating input power (Pin = Vin × Iin).

In the test flow, this supply replaces the transformer/rectifier stage during Phase 3 (open-loop bring-up) and provides a controlled DC bus during automated regulation tests.

4. Electronic Load (ElectronicLoad)

Purpose: Apply a precise, programmable load to the power supply output to simulate real-world current draw.

Methods exposed:

- set_mode("CURRENT") — Selects constant-current mode (also supports VOLTAGE, RESISTANCE, POWER).
- set_current(current) — Programs the load current (e.g., 0.5 A, 2.5 A, 5.0 A).
- input_on / input_off — Connects or disconnects the load from the DUT.
- measure_voltage / measure_current — Reads the actual voltage across and current through the load terminals, used to compute output power (Pout = Vout × Iout) and efficiency.

This is the core instrument for load regulation, efficiency mapping, and transient response testing.

5. Digital Multimeter (DMM)

Purpose: Provide high-accuracy voltage and resistance measurements independent of the power supply or load internal meters.

Methods exposed:

- configure_dc_voltage(range_val) — Sets the measurement range (e.g., 10 V or 20 V) for optimal resolution.

• read() — Triggers a measurement and returns the floating-point value.
• configure_4wire_resistance() — Switches to 4-wire (Kelvin) resistance mode for accurate shunt resistor or trace resistance measurements.

The DMM serves as the 'golden reference' for output voltage accuracy and ripple measurements, since its precision typically exceeds that of the power supply or e-load readbacks.

6. Oscilloscope (Oscilloscope)

Purpose: Capture and analyze time-domain waveforms for gate drive, switching node, ripple, and transient events.

Methods exposed:

• autoscale() — Automatically adjusts vertical and horizontal scales.
• set_timebase(scale) — Sets horizontal scale in seconds per division (e.g., 100e-6 for 100 µs/div).
• set_channel_scale(ch, scale) — Sets vertical sensitivity per channel (e.g., 0.5 V/div for output ripple).
• measure_vpp / measure_frequency — Queries built-in automated measurements.
• run / stop / single — Controls acquisition mode; single is used for one-shot transient capture.
• get_waveform(ch) — Downloads the raw ADC sample buffer as a Python byte array, enabling custom post-processing (FFT, peak detection, settling-time calculation).

This class is essential for verifying gate-drive dead-time, SW node ringing, and load-transient overshoot.

7. STM32 Serial Telemetry Reader (STM32Telemetry)

Purpose: Receive real-time internal state data from the STM32 microcontroller over UART while tests run.

Architecture:

- Opens a serial port (e.g., /dev/ttyUSB0 or COM3) at 115200 baud.
- Spawns a daemon thread that continuously reads lines from the UART in the background.
- Parses CSV telemetry lines in the format: Vset,Vmeas,Imeas,Duty,Temp,Status
- Stores the latest parsed dictionary in self.latest and pushes it into a thread-safe queue.
- get_latest() returns the most recent telemetry snapshot without blocking the test script.

This gives the Python test harness visibility into the firmware's PID duty cycle, ADC readings, and fault flags — data you cannot get from external instruments alone.

8. STM32 Firmware Telemetry Transmitter (C Code)

Purpose: Generate the CSV telemetry stream on the STM32 side that the Python reader consumes.

The SendTelemetry() function formats six variables into a single printf() line:

- v_setpoint — The user-requested output voltage.
- ADC_ReadVoltage() — The ADC-measured output voltage (feedback for the PID loop).
- INA226_ReadCurrent() — The high-side current reading from the I^2C current sensor.
- duty_cycle — The active PWM duty cycle (0–100%).
- ADC_ReadTemp() — The NTC thermistor or internal die temperature.
- fault_flag — A binary OK/FAULT status for protection events.

Called at ~10 Hz from a timer or the main loop, this bridges the firmware and test automation worlds.

9. Complete Automated Test: Load Regulation

Purpose: Automatically execute the Phase 5A load regulation test: fix output at 12 V, sweep load from 0 A to 5 A in 0.5 A steps, and log all data.

Execution flow:

1. Initialize PSU, e-load, DMM, and optional STM32 telemetry reader.
2. Reset all instruments and configure the DMM for 20 V DC range.
3. Set the bench supply to 45 V / 2 A limit and enable output (simulates rectified DC bus).
4. Loop through load currents [0, 0.5, 1.0, ..., 5.0] A:

    a. Set e-load current and enable input.
    b. Wait 500 ms for settling.
    c. Read Vout from DMM, Vin/Iin from PSU, Vout/Iout from e-load.
    d. Compute Pin, Pout, and efficiency.
    e. Capture STM32 telemetry (setpoint, measured voltage, duty cycle).
    f. Append everything to a pandas DataFrame.
    g. Print a status line and flag out-of-spec regulation (> ±0.5%).

5. Export results to CSV.
6. Generate two matplotlib plots:

    a. Load Regulation: Vout vs. Iload with ±0.5% tolerance bands.
    b. Efficiency Curve: η vs. Iload with 88% target line.

7. Safely shut down all instruments in a finally block.

This single script replaces hours of manual knob-turning and notebook logging.

10. Automated Transient Response Capture

Purpose: Capture the output voltage deviation when the load steps abruptly (e.g., 0 A → 5 A) to validate dynamic performance.

Execution flow:

1. Configure the oscilloscope for 100 µs/div timebase and appropriate channel scales.
2. Arm the scope in SINGLE acquisition mode with a falling-edge trigger on Vout at 11.5 V.
3. Turn on the e-load at the initial current (e.g., 0 A).
4. Command the e-load to step to the final current (e.g., 5 A).
5. Wait briefly, then stop the scope and download the waveform buffer.
6. Convert raw bytes to a numpy array and build a time axis.
7. Plot the transient waveform, mark the load-step instant, and save as PNG.

Pass criteria derived from this capture: settling time < 1 ms and overshoot < 5% (per Phase 5B).

11. Automated Protection Testing (OVP)

Purpose: Inject an over-voltage fault and measure the firmware's response time.

Execution flow:

1. Configure the oscilloscope for 10 µs/div to resolve fast protection events.
2. Start the STM32 telemetry reader.
3. Send a serial command "SETV 35.0" to the STM32, commanding an output above the 32 V OVP threshold.
4. Wait 100 ms and check telemetry status.
5. Verify the status flag reads FAULT and the output has collapsed to ~0 V.

The scope (armed in single mode) captures the exact timing between the fault injection and the PWM shutdown, verifying the < 20 µs response requirement.

12. Full Regression Test Suite (pytest)

Purpose: Structure all tests into a formal pytest framework for unattended, repeatable pass/fail validation.

Structure:

- A pytest fixture (scope="class") initializes the instrument suite once and yields it to all test methods.
- test_voltage_accuracy_12V — Asserts DMM reading is within ±0.5% of 12 V target.
- test_load_regulation_5A — Sets 5 A load and asserts Vout stays within ±0.5%.
- test_efficiency_full_load — Computes and asserts η > 88%.

The fixture's teardown (after yield) safely turns off the load and PSU regardless of pass/fail status. Running pytest -v produces a clean report with green checkmarks or red failures for every requirement.

13. Data Logging & Report Generation

Purpose: Convert raw test data and plots into a professional Microsoft Word test report automatically.

Execution flow:

1. Create a new Document object.
2. Add a title heading.
3. Build a formatted table with three columns: Parameter, Spec, Measured.
4. Compute summary statistics from the pandas DataFrame:
    a. max_eff — highest efficiency recorded.
    b. min_v — lowest output voltage at full load.
    c. load_regulation_percent — deviation from 12 V target.
5. Populate table rows with formatted values.
6. Insert the matplotlib plot image below the table.
7. Save the document as Test_Report.docx.

This eliminates manual copy-paste from spreadsheets into Word and ensures every report is generated from the same raw data with zero transcription errors.

Summary: Test Automation Architecture

The thirteen code blocks form a complete vertical stack:

- • Instrument discovery and abstraction (Blocks 1–6) give Python control over every piece of bench hardware.
- • Firmware telemetry (Blocks 7–8) closes the loop between the STM32's internal state and the test harness.
- • Automated test scripts (Blocks 9–11) execute the validation plan from the Testing & Validation Framework document without human intervention.
- • Regression framework (Block 12) turns ad-hoc scripts into repeatable, version-controlled quality gates.
- • Report generation (Block 13) produces deliverable documentation from raw data in seconds.